



UNIVERSITÀ DEGLI STUDI DI PALERMO
DIPARTIMENTO DI INGEGNERIA
CORSO DI LAUREA IN INGEGNERIA INFORMATICA

System Design Document

SPECIFICHE DI SISTEMA DEL SOFTWARE

ArtID



Team di Sviluppo

Andrea Sciortino
Davide Alaimo
Gabriele Iovino
Giovanni Luca Cusano

Supervisore

Prof.ssa Valeria Seidita

Anno Accademico

2025 - 2026

Indice Generale

1	Introduzione	2
1.1	Obiettivi del sistema	2
1.2	Obiettivi di progettazione	2
1.2.1	Affidabilità, Robustezza e Continuità del Servizio	2
1.2.2	Sicurezza e Integrità dei Dati	2
1.2.3	Prestazioni ed Efficienza	3
1.2.4	Usabilità ed Esperienza Utente	3
1.2.5	Estendibilità e Manutenibilità	3
1.3	Definizioni	3
2	Architettura software attuale	4
3	Architettura software proposta	5
3.1	Panoramica Architetturale	5
3.1.1	Decomposizione Architetturale in Macro-Sottosistemi	5
3.2	Identificazione dei Sottosistemi Funzionali Cross-Platform	5
3.3	Decomposizione in Sottosistemi	7
3.3.1	Sottosistema di Autenticazione	8
3.3.2	Sottosistema di Gestione Profilo	8
3.3.3	Sottosistema di Gestione Materiali	8
3.3.4	Sottosistema di Condivisione	8
3.3.5	Sottosistema di Discovery	9
3.4	Mappatura degli oggetti nei sottosistemi	10
3.4.1	Autenticazione	10
3.4.2	Gestione Profilo	11
3.4.3	Gestione Materiali	12
3.4.4	Condivisione	13
3.4.5	Discovery	14
3.5	Mappatura Hardware/Software	15
4	Gestione dei Dati Persistenti	16
4.1	Diagramma E-R	16

1 Introduzione

1.1 Obiettivi del sistema

Il progetto “ArtID” nasce con lo scopo di ingegnerizzare e rilasciare una piattaforma software centralizzata espressamente dedicata agli studenti delle istituzioni appartenenti al comparto AFAM (Alta Formazione Artistica, Musicale e Coreutica). L’applicativo si propone come l’ambiente digitale di riferimento per la creazione, la personalizzazione e l’integrazione dell’identità digitale e del percorso formativo-artistico dei discenti.

L’obiettivo cardine del sistema consiste nella digitalizzazione standardizzata, nella valorizzazione e nella centralizzazione del patrimonio artistico e accademico individuale. Lo strumento mira a superare la frammentazione delle informazioni, aggregando in un unico ecosistema protetto sia i dati anagrafici dello studente, sia le sue certificazioni, sia la produzione artistica multimediale generata durante la carriera. Il sistema si rivolge primariamente agli studenti in qualità di amministratori unici del proprio portfolio, e secondariamente a una platea di attori esterni (commissioni d’esame, direttori artistici e istituzioni partner) abilitati alla consultazione selettiva del materiale condiviso.

Nello specifico, il sistema persegue i seguenti traguardi strategici e operativi:

- **Centralizzazione e Unificazione:** Abbattere la dispersione dei dati eliminando la necessità di ricorrere a molteplici repository commerciali esterni o a supporti fisici locali, offrendo un unico punto di persistenza governato dall’istituzione AFAM.
- **Autonomia e Flessibilità Configurativa:** Garantire all’utente finale la piena sovranità sui propri dati, permettendo la strutturazione di portfolio dinamici e la modulazione granulare dei livelli di visibilità dei contenuti in base al contesto d’uso.
- **Disseminazione Sicura e Controllata:** Abilitare canali di condivisione esterni protetti e monitorabili, capaci di fornire metriche analitiche oggettive circa l’avvenuta consultazione dei materiali condivisi.
- **Compliance e Scalabilità Architetture:** Configurare un’infrastruttura tecnologica aderente alle vigenti normative in materia di protezione dei dati personali (GDPR) e predisposta nativamente per future integrazioni con i nodi di identità federata nazionali ed europei.

1.2 Obiettivi di progettazione

1.2.1 Affidabilità, Robustezza e Continuità del Servizio

- **Disponibilità e Uptime:** Il sistema e il DBMS devono essere costantemente in esecuzione all’interno del nodo AFAM in cui sono installati, per permettere agli utenti di interagire con il sistema principale in ogni momento. È tuttavia tollerabile un periodo di inattività di pochi minuti al giorno all’infuori degli orari di picco (scolastici e/o lavorativi) per permettere interventi di manutenzione. L’uptime deve essere il più vicino possibile al 100%.
- **Gestione della Connettività ed Errori:** Il sistema deve essere in grado di rispondere adeguatamente a una perdita di connessione, inizialmente ritentando il collegamento. Se il problema persiste, deve segnalarlo all’utente e assicurare la coerenza e la persistenza dei dati immagazzinati anche in caso di errore. In caso di interruzione, il sistema deve permettere all’utente di riprendere il lavoro dal punto in cui si era interrotto.

1.2.2 Sicurezza e Integrità dei Dati

- **Protezione e Sviluppo Sicuro:** Il sistema deve essere protetto da vulnerabilità note, adottando le pratiche conosciute per uno sviluppo sicuro. Non deve permettere all’utente di inserire input invalidi o sintatticamente scorretti, implementando meccanismi di validazione e sanificazione dell’input e l’uso di query parametrizzate.
- **Crittografia delle Credenziali:** Le password degli utenti devono essere memorizzate criptate all’interno

del database.

- **Transazionalità:** Le richieste al DBMS devono essere gestite in modo transazionale per garantire l'integrità dei dati.

1.2.3 Prestazioni ed Efficienza

- **Tempi di Risposta:** Il sistema deve poter rispondere alle richieste dell'utente entro cinque secondi, non considerando il tempo di trasferimento dei file multimediali.
- **Ottimizzazione del DBMS:** Il sistema deve effettuare il minimo numero possibile di interrogazioni al DBMS per rispondere a una data richiesta.

1.2.4 Usabilità ed Esperienza Utente

- **Interfaccia Intuitiva:** L'utilizzo del sistema deve essere intuitivo e immediato, pensato anche per utenti dalle scarse competenze tecnologiche. Il sistema deve minimizzare dove possibile le interazioni con l'utente necessarie per portare a termine una data operazione.

1.2.5 Estendibilità e Manutenibilità

- **Modularità dell'Identità Digitale:** Il sistema deve essere progettato per essere in grado di accogliere diversi moduli di identità federata (SPID/eIDAS) senza alterare la logica di business.

1.3 Definizioni

Proprietà	Descrizione
Membro AFAM	Individuo autenticato che utilizza il software.
Guest	Individuo esterno all'istituzione AFAM o individuo non autenticato che interagisce con il software.
Materiale	Qualsiasi file che faccia parte della produzione artistica dello studente, come registrazioni audio, video, spartiti e curriculum utili a rappresentarne le competenze e l'esperienza.
ArtID	Contenitore dinamico che contiene i materiali del membro AFAM e che può essere condiviso con altri membri AFAM o con utenti esterni.
Condivisione interna	Condivisione di un ArtID da un Membro a un altro.
Condivisione esterna o Link	Condivisione di un ArtID tramite link tra un Membro e un Guest.
DBMS	Sistema per la gestione dei dati permanenti degli ArtID e dei Membri
Codice OTP	Codice numerico monouso generato dal sistema e inviato al Membro tramite posta elettronica. È formato da 6 cifre ed ha una validità temporale limitata (5 minuti dalla generazione), oltre la quale decade e non può più essere utilizzato. Garantisce che l'indirizzo email fornito dal Membro in fase di registrazione sia reale, attivo e sotto il suo diretto controllo.

2 Architettura software attuale

Al momento non esiste alcuna architettura software stabilita che ricopre il ruolo che ArtID si propone di ricoprire.

Tutte le funzionalità offerte dal sistema sono infatti al momento gestite manualmente e senza supporto di software dedicati, appoggiandosi al più a repository esterni o file storage locali per la memorizzazione dei contenuti multimediali.

3 Architettura software proposta

3.1 Panoramica Architeturale

Per il sistema è stato scelto uno stile architeturale **Client-Server disaccoppiato**, basato sul protocollo di comunicazione stateless HTTP. Il sistema è suddiviso in due macro-sottosistemi principali:

- **Sottosistema Client:** Front-end eseguito come Single Page Application (SPA) accessibile da browser, strutturato su due livelli: il livello **Interfaccia** e il livello **Controller**.
- **Sottosistema Server:** Back-end Java organizzato internamente secondo i principi di una **Architettura a Layer** (*REST Controller* → *Service* → *DAO/Repository*).

L'adozione del paradigma **Client-Server** è dettata dalla necessità di garantire una netta **separazione tra l'utente finale e i dati con cui questo interagisce**. Questi ultimi, per la natura stessa dell'applicazione, devono essere centralizzati, controllati e protetti.

La scelta di un'architettura rigida a livelli, in totale 5 tra Client e Server, è analogamente motivata dall'obiettivo di esercitare un rigido controllo sul flusso dei dati sensibili e garantire la modularità del codice.

Infine si è deciso di utilizzare il pattern **Repository** per un ulteriore livello di astrazione nell'accesso ai dati permanenti, nascondendo la reale implementazione del database ai livelli soprastanti.

3.1.1 Decomposizione Architeturale in Macro-Sottosistemi

3.1.1.1 Macro-Sottosistema Client-Side

Il lato Client si sviluppa su due livelli logici:

- **Livello Interfaccia (Boundary):** Composto dalle viste grafiche. Si occupa esclusivamente del rendering visivo, della cattura degli eventi utente e della gestione dello stato locale della UI.
- **Livello Controller (Control Client-Side):** Incapsula la logica applicativa locale. Intercetta le azioni dell'interfaccia, coordina lo stato globale della UI, effettua una prima fase di validazione degli input utente e invoca le API REST per la sincronizzazione dei dati con il server.

3.1.1.2 Macro-Sottosistema Server-Side

Il lato Server elabora le richieste provenienti dal client attraverso tre strati logici rigidi:

- **REST Controller Layer:** Punto di ingresso unico per le richieste HTTP. Gestisce l'instradamento (routing), l'interpretazione delle richieste (deserializzazione dei dati) e l'emissione delle risposte HTTP.
- **Service Layer (Control Server-Side):** Nucleo computazionale che ospita la componente Server-Side degli oggetti **Control** originariamente identificati nel RAD. Esegue le regole di business e coordina le transazioni applicative.
- **DAO/Repository Layer:** Isola la logica di business dalle specifiche infrastrutturali di memorizzazione, gestendo l'interazione diretta con il database.

3.2 Identificazione dei Sottosistemi Funzionali Cross-Platform

Sebbene la decomposizione precedentemente introdotta separi fisicamente i componenti in base all'ambiente di esecuzione (*Client* nel browser e *Server* sulla JVM), dal punto di vista logico il sistema è organizzato in **Sottosistemi Funzionali**. I macro-casi d'uso individuati in fase di analisi (*Gestione profilo*, *Condivisioni*, *Discovery*, *Gestione Materiali*, *Autenticazione*) si ripresentano come sottosistemi verticali che

attraversano l'intera architettura a livelli.

Le funzionalità di ciascun sottosistema sono quindi fisicamente distribuite ed eseguite in modo collaborativo tra Client e Server. Ogni sottosistema funzionale attraversa i vari strati secondo una catena distributiva, qui esemplificata evidenziando la corrispondenza tra gli oggetti individuati nel RAD e il rispettivo livello di design:

1. **Layer di Presentazione Client:** Corrisponde alla parte **Interfaccia** del sottosistema, contiene gli oggetti *Boundary*.
2. **Layer di Controllo Client:** Corrisponde alla parte **Controller** del sottosistema lato Client, contiene gli oggetti *Control (Client-side)*.
3. **Layer di Controllo Server (Service Layer):** Corrisponde alla parte **Controller** del sottosistema lato Server, contiene gli oggetti *Control (Server-side)*.

I due ulteriori livelli del Server, ovvero **REST Controller** e **DAO/Repository**, non corrispondono direttamente a oggetti logici individuati nella fase di analisi del RAD. Essi svolgono un ruolo prettamente architetturale emerso in questa fase di analisi architetturale, con lo scopo di isolare il livello di Controllo lato Server (Service) rispettivamente dall'interazione HTTP con il Client (**REST Controller**) e dall'implementazione fisica della persistenza dei dati (**DAO/Repository**).

A livello di deployment, il **lato Server del sistema e il DBMS dovranno essere installati stabilmente presso il nodo dell'istituzione AFAM**, garantendo il controllo istituzionale e centralizzato sui dati. Al contrario, il **lato Client del sistema potrà essere eseguito su un numero arbitrario di nodi indipendenti**, quali i dispositivi personali degli utenti finali.

3.3 Decomposizione in Sottosistemi

Di seguito la rappresentazione della suddivisione in sottosistemi del sistema proposto:

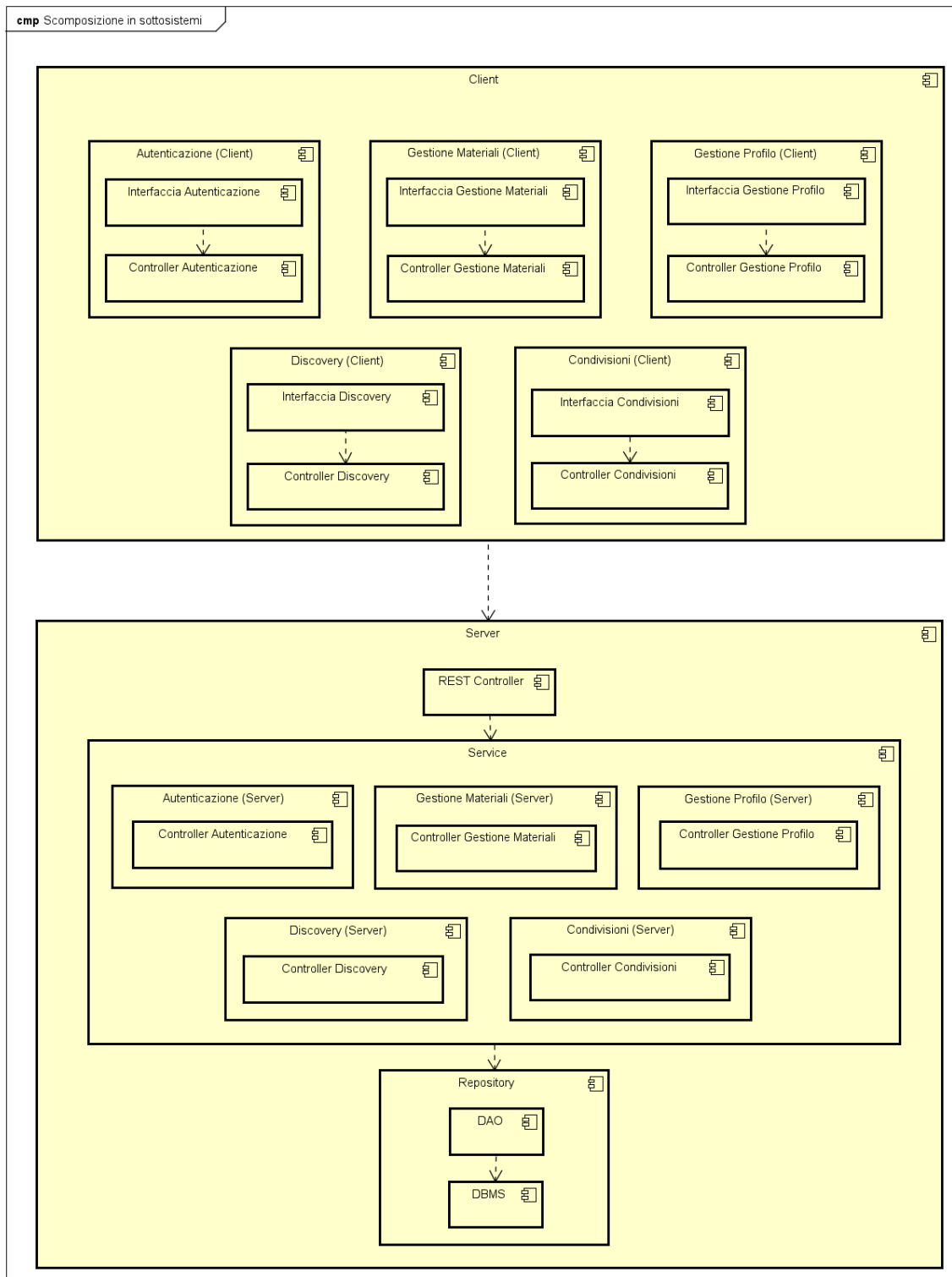


Figure 1: Scomposizione in sottosistemi

3.3.1 Sottosistema di Autenticazione

Questo sottosistema è responsabile della sicurezza, dell'identificazione e del controllo degli accessi di tutti gli utenti all'interno della piattaforma.

- **Responsabilità Principali:**

- Gestione del ciclo di vita delle sessioni utente (Login, Logout e Registrazione dei Membri).
- Integrazione con i gateway ufficiali SPID per la verifica certificata dell'identità digitale.
- Generazione, invio e validazione di codici OTP (*One-Time Password*) per i meccanismi di autenticazione a due fattori.
- Coordinamento delle procedure di recupero o modifica delle credenziali di accesso.
- Gestione della procedura di chiusura e cancellazione definitiva dell'account in conformità con le normative vigenti sulla protezione dei dati.

3.3.2 Sottosistema di Gestione Profilo

Questo sottosistema gestisce il ciclo di vita dei dati anagrafici, di contatto e formativi del Membro AFAM. Si occupa inoltre della persistenza e della gestione dei Certificati caricati dall'utente. Gestisce inoltre le preferenze sulla privacy.

- **Responsabilità Principali:**

- Manutenzione e aggiornamento delle informazioni personali e di contatto.
- Amministrazione delle impostazioni di visibilità del profilo e delle preferenze sulla ricezione delle condivisioni.
- Gestione completa (caricamento, modifica, consultazione ed eliminazione) dei Certificati e dei relativi file scaricabili associati.

3.3.3 Sottosistema di Gestione Materiali

Questo sottosistema fornisce i metodi per la costruzione e manutenzione del portfolio digitale del Membro AFAM, costituito dagli *ArtID* e dai Materiali in essi contenuti. Controlla l'archiviazione logica dei Materiali e la loro gestione, nonché la gestione degli *ArtID*.

- **Responsabilità Principali:**

- Coordinamento delle operazioni di upload, catalogazione, filtraggio e rimozione e aggiunta ai preferiti dei Materiali.
- Strutturazione dell'identità artistica tramite la creazione e configurazione degli *ArtID*, inclusa l'associazione di metadati quali descrizione e immagine e l'assegnazione di Tag descrittivi e l'aggiunzione dei Materiali.
- Gestione dei livelli di visibilità dei singoli *ArtID*.

3.3.4 Sottosistema di Condivisione

Questo sottosistema gestisce la creazione e la gestione delle Condivisioni verso l'esterno o verso altri utenti della piattaforma, nonché la gestione delle relative notifiche (tramite email) di presa visualizzazione o ricevuta condivisione.

- **Responsabilità Principali:**

- Condivisione mirata degli *ArtID* verso specifici Membri registrati attraverso Condivisioni Interne.
- Generazione di Condivisioni Esterne costituite da token temporanei e link da poter condividere con Guest, con possibilità di impostare descrizioni e date di scadenza.
- Tracciamento dei contatori di visualizzazione dei link esterni e supporto alla modifica, disattivazione o revoca dei permessi di accesso.

3.3.5 Sottosistema di Discovery

Questo sottosistema costituisce il motore di ricerca e consultazione pubblica della piattaforma, pensato per l'esplorazione rapida da parte di Guest. Espone i contenuti in modalità di sola lettura, applicando rigidi filtri di sicurezza per rispettare le impostazioni di privacy definite dai Membri.

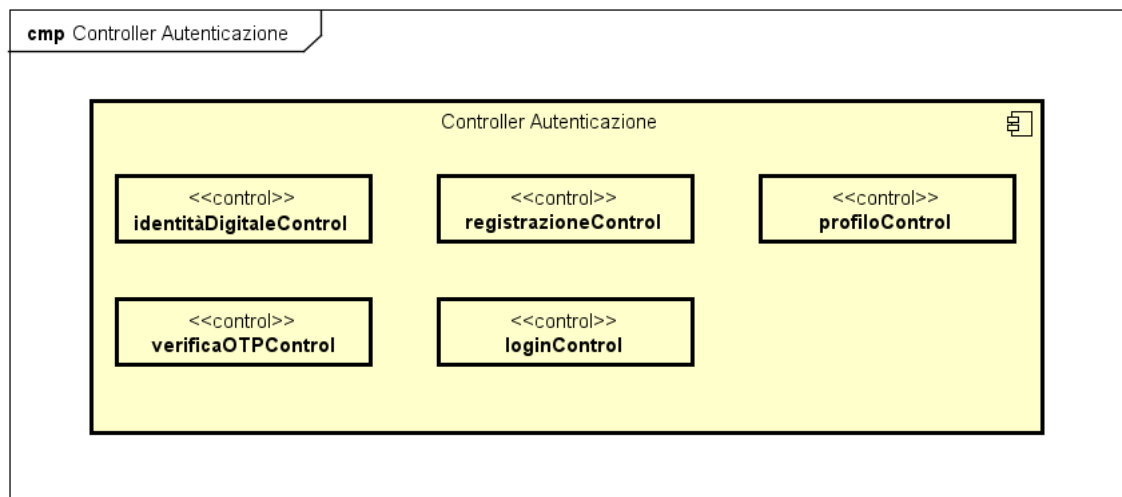
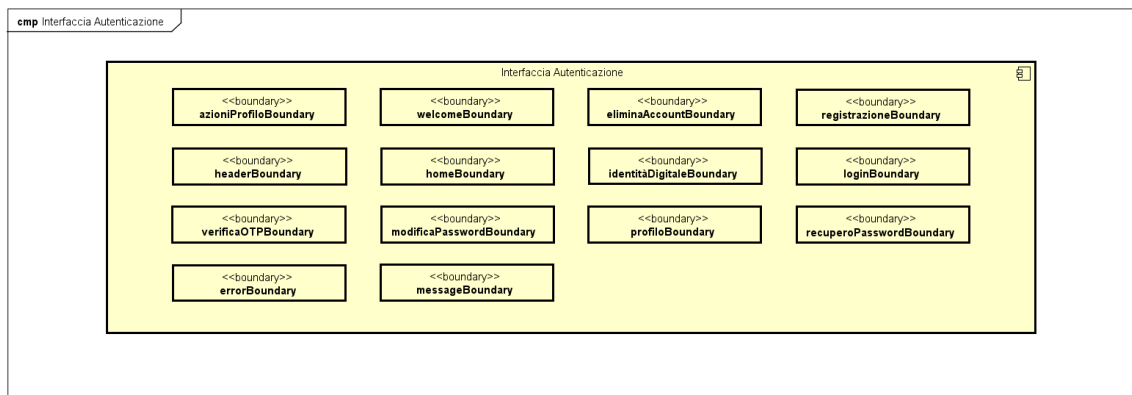
- **Responsabilità Principali:**

- Fornitura di funzionalità di ricerca testuale (anche per corrispondenza parziale del nome) per l'individuazione dei Membri dando la possibilità di consultarne il Profilo o ArtID da questi creati.
- Controllo dei permessi di visualizzazione del Profilo o dei singoli ArtID, esponendo solamente quello che il Membro ha contrassegnato come Pubblico.

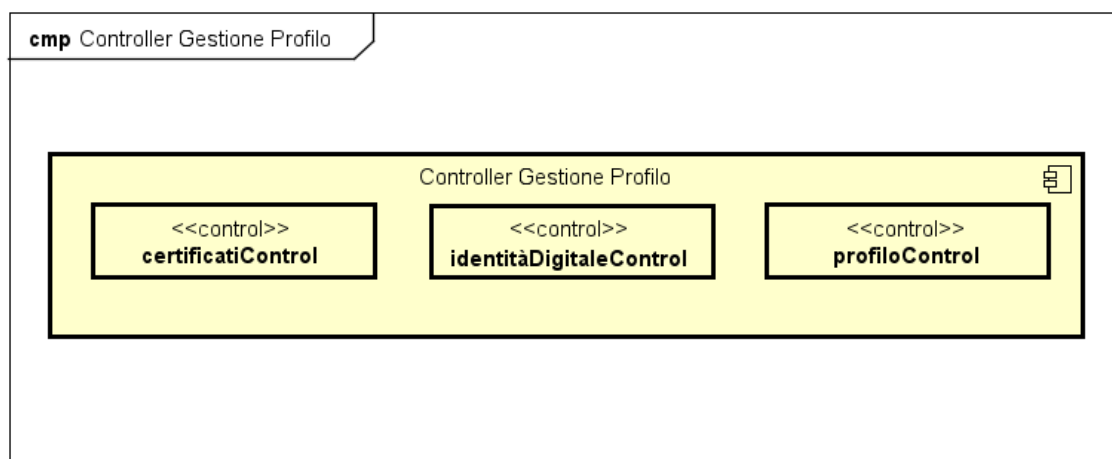
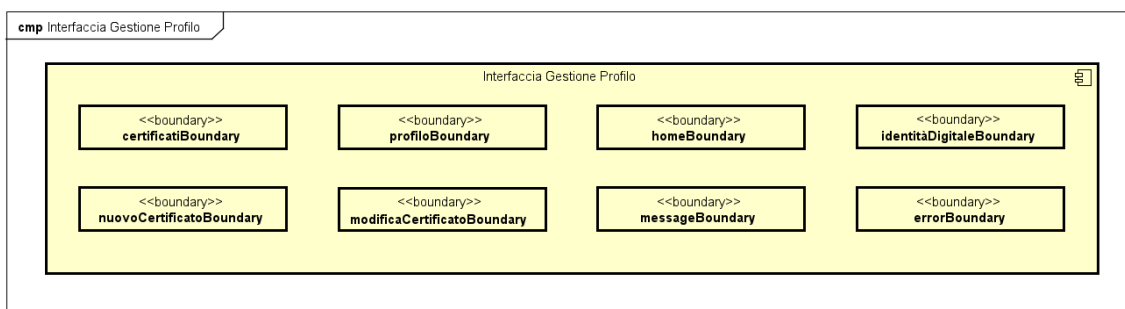
3.4 Mappatura degli oggetti nei sottosistemi

Di seguito sono riportate le funzionalità offerte dai sottosistemi:

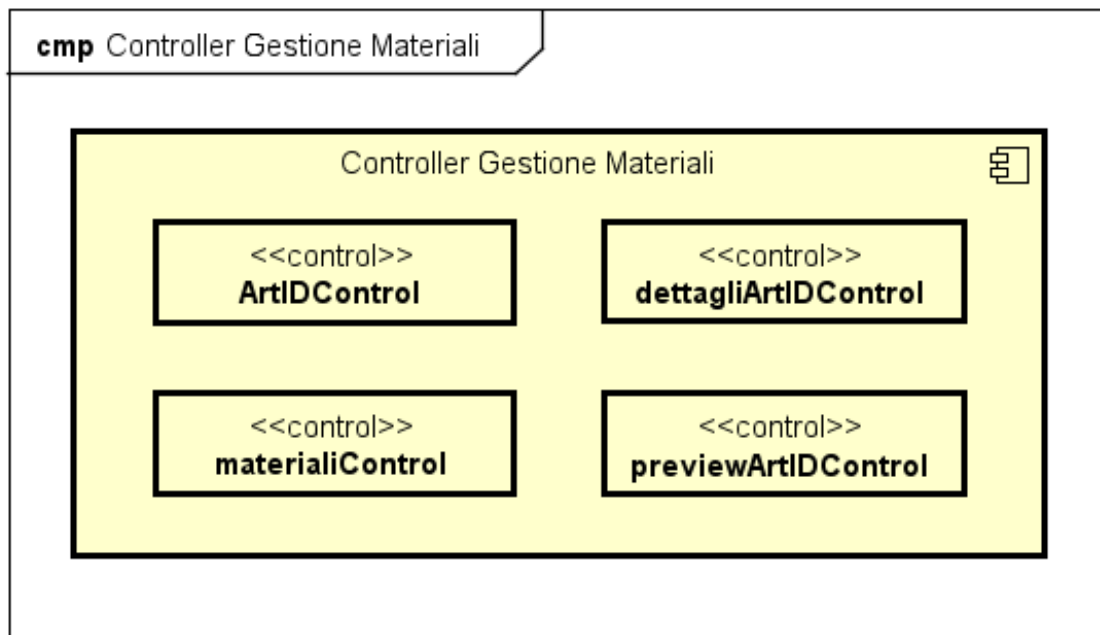
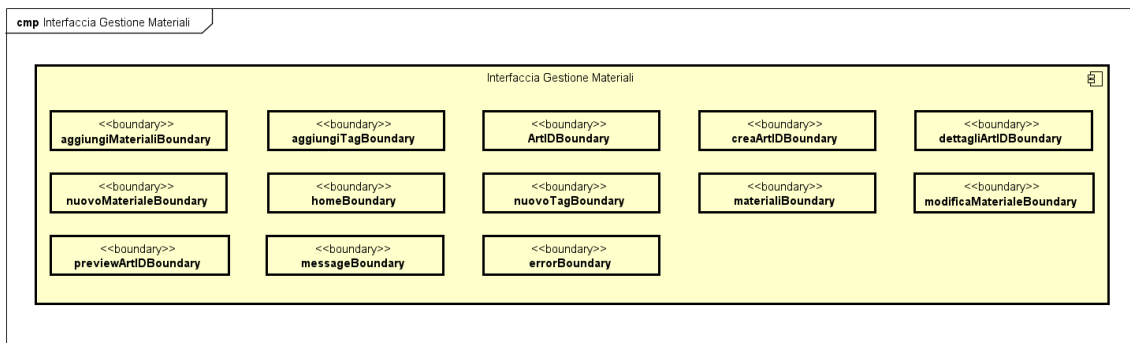
3.4.1 Autenticazione



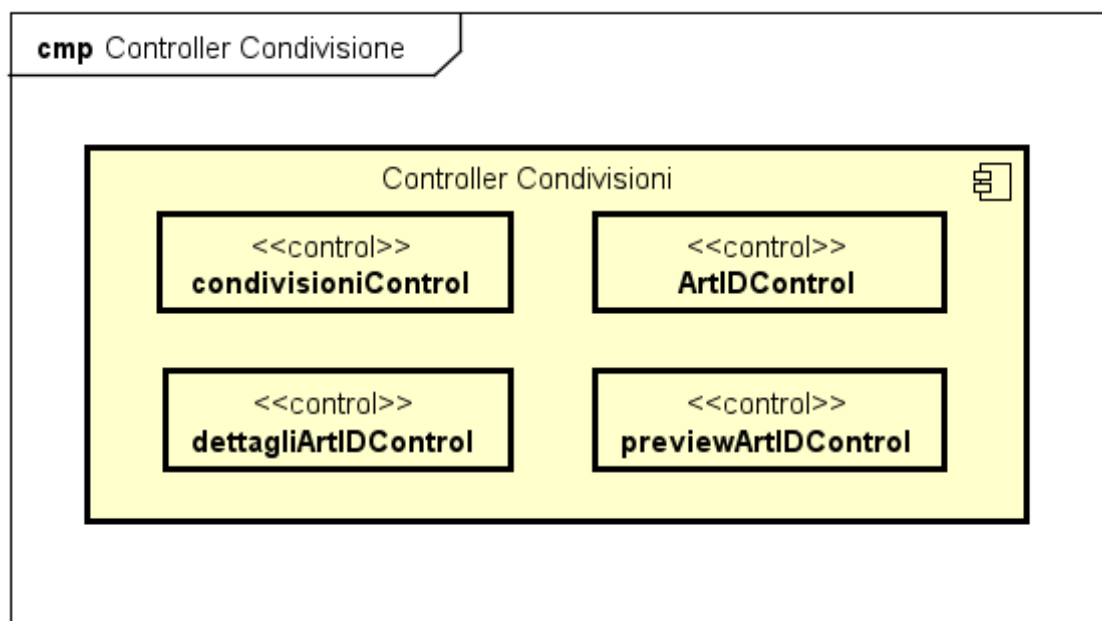
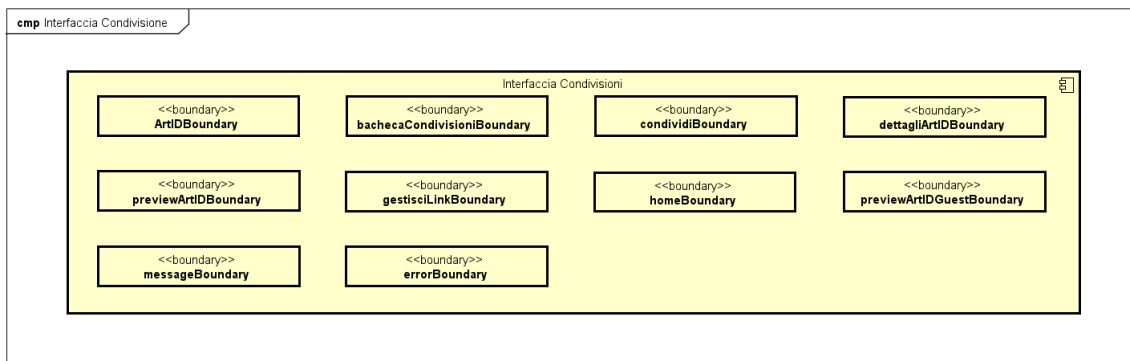
3.4.2 Gestione Profilo



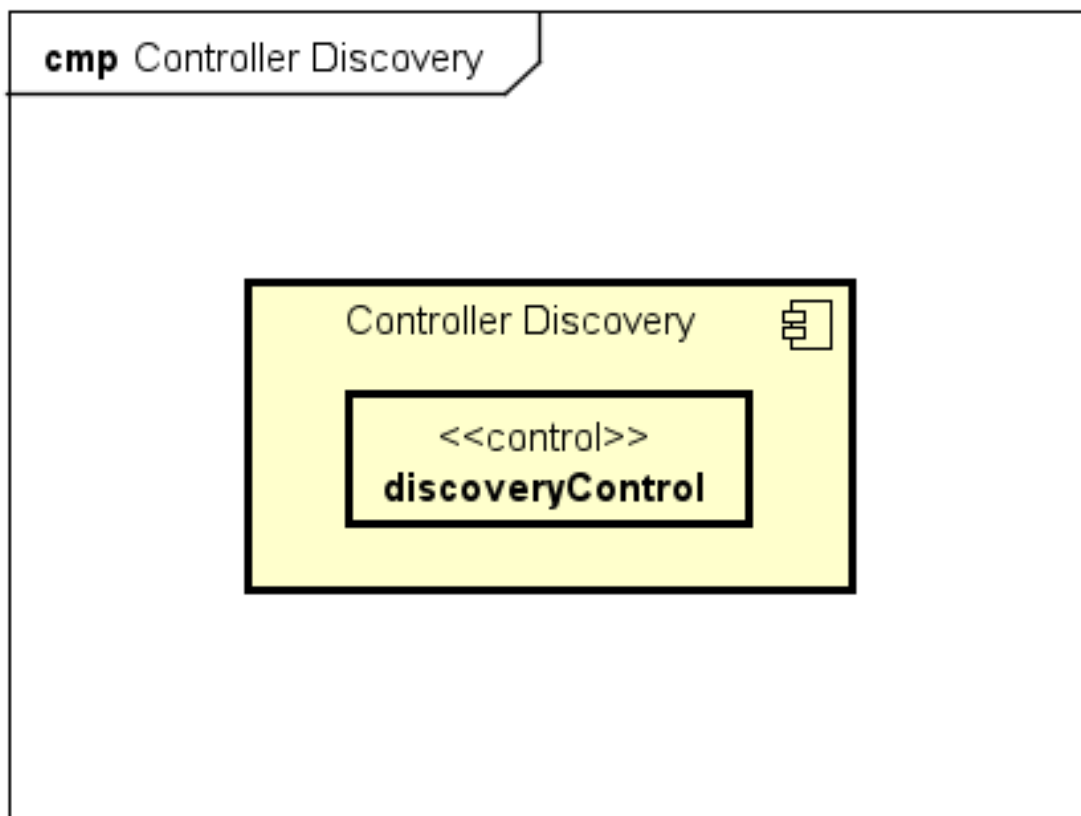
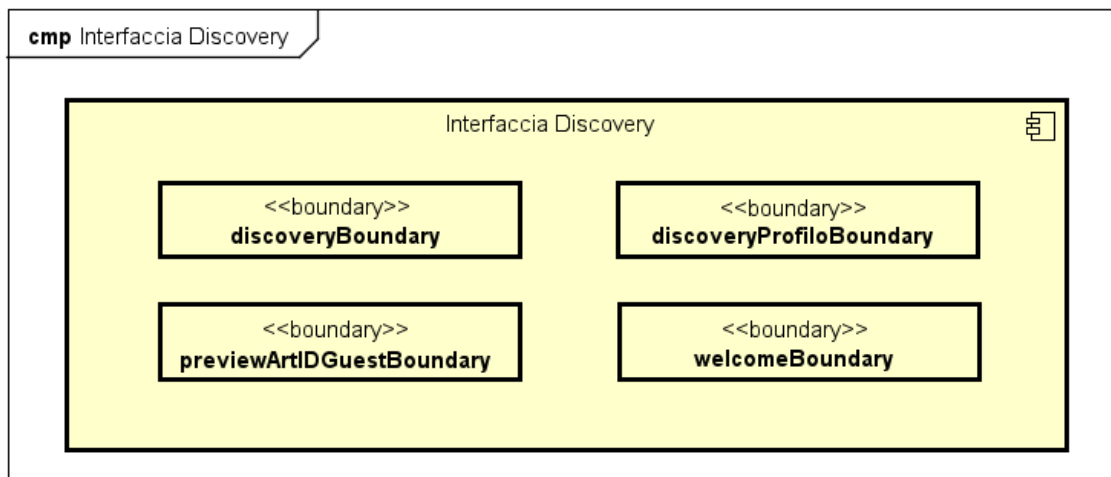
3.4.3 Gestione Materiali



3.4.4 Condivisione



3.4.5 Discovery



3.5 Mappatura Hardware/Software

La mappatura dei componenti software sui nodi hardware del sistema riflette la separazione strutturale introdotta dallo stile Client-Server e dal pattern MVC. Il sistema è distribuito su due macro-nodi principali:

- **Nodo “Dispositivo Utente” (Client):** Ospita il sottosistema *View* e le componenti di interfaccia del *Controller*. Questo nodo hardware (rappresentato dai dispositivi personali degli utenti) esegue l'applicazione client, gestendo l'interazione diretta con l'utente e l'invio delle richieste verso il server centralizzato.
- **Nodo Centralizzato “Server AFAM”:** Rappresenta il nucleo del sistema e ospita la logica di business principale (*Controller* di backend), l'architettura *Repository* e il *DBMS*. Questo nodo gestisce la sicurezza, la persistenza transazionale dei dati all'interno della sorgente dati e l'elaborazione dei portfolio multimediali, esponendo le API necessarie per permettere ai nodi client di comunicare e recuperare o inserire informazioni in modo sicuro.

Le comunicazioni tra il nodo client e il server centrale avvengono tramite canali di rete protetti, isolando completamente l'accesso diretto al DBMS dall'ambiente esterno a garanzia della sicurezza dei dati istituzionali.

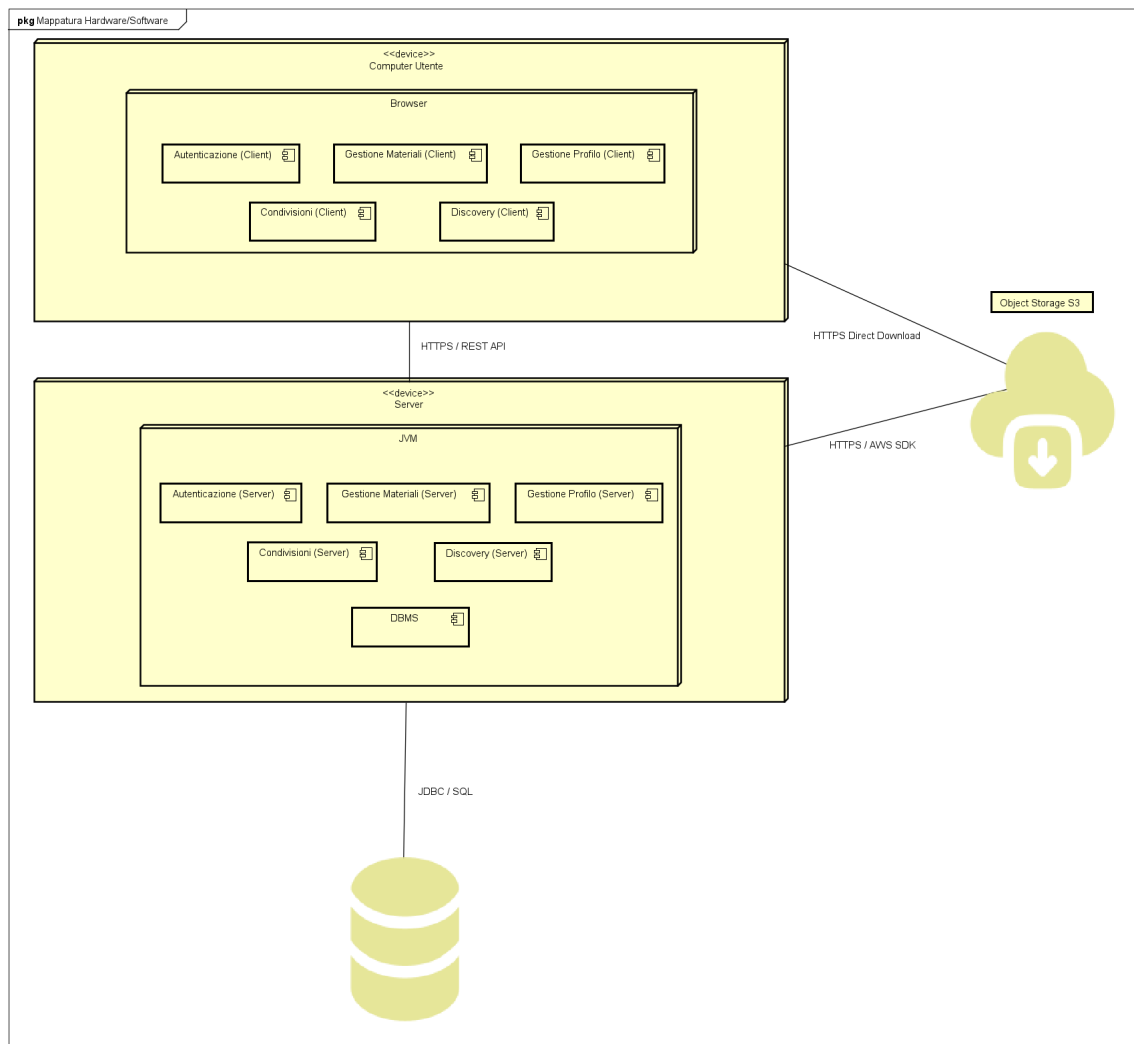


Figure 2: Mappatura Hardware/Software

4 Gestione dei Dati Persistenti

Il livello di memorizzazione è realizzato su un **DBMS relazionale** interrogato tramite **interfaccia SQL**, che assicura il rispetto delle proprietà ACID. La permanenza dei file è affidata a un object storage esterno.

4.1 Diagramma E-R

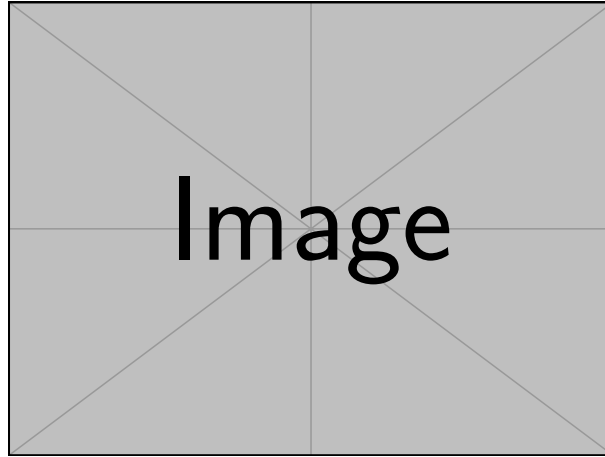


Figure 3: This is a built-in placeholder image.